PQC Migration Readiness Framework — Codebase Analysis & Research Summary

Version: v1.0.0 (Phase 1 release) · **Date:** 2026-07-09 · **Repository:** github.com/Arpan0995/pqc-migration-readiness

Executive summary. This is a research framework, in Java (JDK 21), for estimating what it would cost to migrate a Java codebase to post-quantum cryptography (PQC) — before anyone commits engineering time to the migration. It contains (a) a static **auditor** that scans Java source for quantum-vulnerable cryptographic usage and produces a per-module difficulty score with ranked, explained hotspots; (b) a runtime **crypto-agility layer** over the JCA/JCE that switches between classical, hybrid, and PQC-only algorithm suites by policy, with capability negotiation and audit logging; (c) a **JMH benchmark harness** whose completed campaign quantifies the runtime cost of PQC and of the agility abstraction itself on the JVM; and (d) **case-study estimation reports** for four real public codebases (3,564 source files). The novel claims worth academic attention are the *structural fragility indicator* taxonomy as an effort predictor, a *classpath-free cross-file constant propagation* technique with measured impact, a *pre-registered scoring model* as a bias control, the empirical finding that *type coupling — not algorithm call sites — dominates* the migration surface of mature Java libraries, and a JVM-specific *allocation/GC-pressure characterization* of PQC that the C/Rust-centric PQC literature does not surface.

1. Research Question & Motivation (what we are testing, and why)

Question. Can static code patterns in a Java codebase predict the actual effort required to migrate it to post-quantum cryptography?

Hypothesis (H1). A difficulty score combining (a) cryptographic API usage patterns and (b) *structural fragility indicators* — hardcoded key/signature buffer sizes, fixed-width serialization, protocol pinning, concrete key-type coupling — correlates with real migration effort better than the naive baseline any linter or inventory tool provides: a raw count of vulnerable call sites.

Why it matters. NIST finalized ML-KEM (FIPS 203), ML-DSA (FIPS 204), and SLH-DSA (FIPS 205) in August 2024; NIST IR 8547 deprecates quantum-vulnerable algorithms (RSA, ECDSA, (EC)DH, EdDSA) after 2030 and disallows them after 2035. Organizations therefore know *what* to migrate to, but no existing tool tells them *how much a migration will cost for their specific codebase*. Existing tooling stops at inventory (CBOMkit / the PQCA Sonar Cryptography plugin) or misuse detection (CogniCrypt, CryptoGuard). Meanwhile nearly all PQC performance literature targets C/C++/Rust; the enterprise JVM — dominant in banking, insurance, and government backends, with production fleets on JDK 8–21 that will not see JDK-native PQC (JDK 24+, hybrid TLS in JDK 27) for years — is nearly unstudied. That combination (validated effort model missing × JVM angle missing) is the research gap.

Phasing (an honest constraint). The project deliberately runs in two phases. **Phase 1 (complete, this release)**: build the instrument, run it on real codebases, and publish score-derived *estimates* — explicitly framed as a transparent heuristic, not a validated prediction. **Phase 2 (deferred)**: validate the score against *measured* migration effort (performed or mined migrations), with Spearman/Kendall correlation against the naive baseline. The pre-registered protocol and a working statistical harness (`analysis/correlate.py`: rank correlations plus a paired-bootstrap confidence interval on delta-rho vs. the baseline) already exist; only the ground-truth effort data does not. The tooling refuses to output "engineer-days" anywhere, because no honest conversion factor exists yet.

What Phase 1 itself tests. Even without effort ground truth, the case-study runs answer real empirical questions: does the detector behave sensibly on mature code (including a negative control)? What does

the quantum-vulnerable surface of real Java libraries actually look like? And what does agility cost at runtime on a JVM?

2. Core Architecture

A multi-module Maven project (JDK 21), 42 main-source Java files plus a stdlib-only Python analysis harness:

Module	Files	Role
`auditor`	24	Static detection + scoring + reporting + CLI
`agility-provider`	15	Runtime crypto-agility: policy, negotiation, hybrid primitives, audit
`benchmarks`	3	JMH overhead matrix across classical/hybrid/PQC
`case-studies`	—	Four pinned public codebases (git submodules) + reports + synthesis
`analysis`	—	Phase-2 correlation harness (pure Python stdlib, self-tested)

Auditor pipeline. `Scanner` runs **two passes** over a source tree: pass 1 parses every file (JavaParser, Java-21 language level; unparseable files are recorded, never fatal) and builds a project-wide `ConstantIndex` of `static final String` constants; pass 2 runs a `DetectionVisitor` per file, collecting *primary findings* (vulnerable API usage, typed by category) and *fragility signals*, each tagged with a lexical **scope key** (enclosing method/constructor, else type). A merge step attaches co-located fragility indicators to the crypto findings they qualify. `ScoringEngine` (deliberately decoupled from detection) aggregates findings into file and module scores under the frozen `ScoreModel` v0, resolves modules via build descriptors (`ModuleResolver`), and emits a `ReadinessReport` rendered by JSON and Markdown writers; a CLI wraps the whole pipeline. Detection is **syntactic and classpath-free by design** — the tool must run on arbitrary codebases nobody builds first.

Agility provider. Two orthogonal axes — `Intent` (key establishment vs. signature) and `Mode` (classical / hybrid / PQC-only) — because harvest-now-decrypt-later makes confidentiality urgent before authenticity. Named `CryptoSuite`s form a single vocabulary shared by policy, negotiation offers, and audit records. A `Negotiator` selects the highest local-preference suite present in the peer's offer, with policy-controlled behavior on failure (fail-closed default, or downgrade bounded by a mode floor). Primitives are delegated to Bouncy Castle; only *composition* is original: `HybridKeyEstablishment` (ephemeral X25519 ECDH + ML-KEM-768 encapsulation, combined as HKDF-SHA256 over the concatenated secrets — the X25519MLKEM768 pattern) and `DualSignature` (ECDSA-P256 + ML-DSA-65, fail-closed AND verification), over a deliberately simple length-prefixed wire framing. Every operation can be recorded to a JSONL audit log (the NIST CSWP 39 evidence-trail requirement).

3. Key Algorithms & Engineering Innovations

(a) Structural fragility indicators (the core novel signal). Beyond flagging vulnerable API calls, the auditor detects code *shapes* that make migration disproportionately expensive, motivated by PQC artifact sizes (ML-DSA-65 signatures are 3,309 B vs. 64–72 B for ECDSA; ML-KEM-768 ciphertexts 1,088 B): **F1** fixed-size buffers at classical sentinel sizes (32, 64, 65, 70, 72, 91, 128, 256, 294, 384, 512 bytes) near crypto dataflow; **F3** protocol/suite pinning (legacy TLS version pins, `setEnabledCipherSuites`/`setEnabledProtocols`); **F4** concrete key-type coupling (APIs typed against `RSAPublicKey`/`ECPrivateKey`/... instead of the `PublicKey`/`Key` interfaces — a change that propagates through every caller); **F6** persisted key material (keystores, X509/PKCS8 encoded key specs — data migration on top of code migration). Indicators attach to co-located findings through the **scope-key merge**,

a cheap, deliberate proxy for dataflow adjacency that avoids full dataflow analysis while keeping false attachment bounded to one method. F2 (fixed-width persistence), F5 (config-sourced algorithm credit), F8 (third-party API boundary) are specified and scoring-ready but not yet detected.

(b) Classpath-free cross-file constant propagation. Real libraries do not write `getInstance("RSA")`; they write `getInstance(KeyUtils.RSA_ALGORITHM)`. Symbol solvers need a compiled classpath — unavailable when scanning arbitrary repositories. The auditor instead exploits the fact that it already holds *the entire source tree*: pass 1 indexes every `static final String` constant with a literal (or literal-concatenation) initializer; resolution then follows local variables (guarded against reassignment), same-file `final` fields, qualified `Type.FIELD` references project-wide, and bare names only when unambiguous across the whole project. Confidence is `HIGH` for direct literals and `MEDIUM` whenever indirection was followed — feeding a planned precision-by-confidence calibration. Measured impact on the four case studies: detected algorithm-selection call sites rose **from 14 to 30 (+114%)**, every recovered site spot-checked as a true positive.

(c) A pre-registered, frozen scoring model as a bias control. `ScoreModel` v0 was frozen in git *before* any effort ground truth exists, so future validation cannot be contaminated by tuning-to-the-answer; any tuned successor (v1) may only be evaluated on codebases not used for tuning. Per finding: difficulty = base weight by category (key establishment 3, signature 3, keygen 2, TLS config 2, JOSE 2, type coupling 1) × the product of fragility multipliers (F1 1.5, F2 2.0, F3 1.5, F4 1.5, F6 2.0, F8 2.5; F5 is a 0.5 *credit*), capped at 6.0. Module score = sum × spread factor (1 + 0.1·log2(1 + files-with-findings)) — forty findings in one file are a focused rewrite; across 25 files they are a campaign. **Urgency** (harvest-now-decrypt-later weight 2.0 vs. signature 1.0) is kept strictly orthogonal to difficulty so validation of the difficulty claim stays clean. Scores map to qualitative **effort tiers** (NONE/LOW/MEDIUM/HIGH/CRITICAL at 0/0.01/10/40/120) — a deliberate refusal to fabricate person-day figures.

(d) Negotiated crypto agility with a single suite vocabulary. The same suite identifiers appear in policy files, capability offers, negotiation results, and audit logs, which makes the audit trail directly interpretable against policy — and makes negotiation itself trivially cheap (measured: nanoseconds; see §6).

(e) Honest-measurement engineering. Two examples that belong in any replication guide: (i) the shaded benchmark uber-jar **silently lost ML-KEM** — Bouncy Castle ships it in a multi-release jar (`META-INF/versions/9/`) that maven-shade flattens — so 6 of 20 benchmark configurations failed with `NoSuchAlgorithmException` and JMH simply omitted them from its JSON; caught by checking result counts against expectation, fixed by running from the module classpath. (ii) The scanned `case-studies/<name>/repo` submodule layout exists because the natural name `target/` is ignored by standard Java `.gitignore` rules.

4. Inputs & Outputs

Auditor. *Input:* any Java source tree — a path is sufficient; no build, no classpath, no configuration. *Output:* `readiness-report.json` (machine-readable; per-module `score`, `tier`, `urgency`, naive-baseline `baselineCount`, LOC, and per-finding `ruleId`, `file:line`, API, algorithm, category, confidence, fragility list, difficulty) and `readiness-report.md` (module ranking table plus ranked hotspots, each with a plain-language *why it is expensive*). Both carry an explicit banner that scores/tiers are Phase-1 heuristics, not validated predictions.

Agility provider. *Input:* an `IntentPolicy` (ordered suite preferences, fail-closed vs. floor-bounded downgrade) and a peer `CapabilityDescriptor`. *Output:* a `NegotiationResult` (selected suite, downgrade flag), the cryptographic operations themselves (length-prefixed wire blobs for hybrid key shares and dual signatures; 32-byte combined secrets), and a JSONL audit stream (timestamp, intent,

mode, suite, peer offer, outcome, duration).

Benchmarks. *Output:* JMH JSON (latency) plus the full run log (per-iteration GC allocation), distilled into `benchmarks/results/RESULTS.md`.

Analysis harness (Phase 2). *Input:* one or more report JSONs joined on (codebase, module) with a measured-effort CSV; *output:* Spearman/Kendall correlations, LOC-controlled partials, and a bootstrap CI on delta-rho vs. the naive baseline. It refuses to run on fewer than three modules of real data and never invents numbers.

5. Verification & Empirical Method (how we tested)

Unit and integration tests — 77 passing (57 auditor, 20 agility-provider), clean `mvn install` on JDK 21. Auditor tests drive the *full pipeline* (temp-dir fixture trees through Scanner, not isolated visitor calls) and include negative controls: symmetric crypto (`AES`, `HmacSHA256`) never flagged; PQC algorithm names (`ML-KEM`) never flagged; fragility signals must *not* leak across method scopes; reassigned variables, non-final fields, and method parameters must *not* be resolved by constant propagation. Agility tests cover the negotiation matrix exhaustively (preference order, fail-closed, floor-bounded downgrade), hybrid round-trips, wrong-recipient and tampered-component rejection (dual-signature AND semantics), and FIPS-size cross-checks (1,088-byte ML-KEM-768 ciphertext, 3,309-byte ML-DSA-65 signature). Testing philosophy: **composition, not primitives** — Bouncy Castle's algorithms are validated upstream against NIST KAT vectors; re-testing them here would add noise, not assurance.

Case-study method. Four public codebases chosen for domain diversity on coarse criteria only (selection cannot leak the fragility structure the score keys on): jjwt 0.13.0 (JWT/JOSE library), Apache Mina SSHD 2.13.1 (SSH protocol; **pinned deliberately pre-PQC** — upstream added ML-KEM hybrids later, preserving a clean Phase-2 mining candidate), Eclipse Californium 3.14.0 (DTLS/IoT), Apache Shiro 2.2.1 (auth framework; expected near-zero asymmetric usage — a designed **negative control**). Each is a git submodule pinned to an exact tag, so every reported number is reproducible against immutable source. Anomalies were investigated rather than reported blind: shiro's zero was verified by direct source inspection (symmetric-only), and every call site recovered by constant propagation was spot-checked as a true positive.

Benchmark method. JMH 1.37, 2 forks, 5×1s warmup, 5×1s measurement per configuration, average-time mode with the GC profiler, on Apple M2 / OpenJDK 21.0.11 / Bouncy Castle 1.84 — 20 configurations across three benchmarks (key-establishment keygen/encapsulate/decapsulate; signature keygen/sign/verify; negotiation under hybrid-capable and classical-only peers). Run from the module classpath (see §3e). Raw JSON and full log are committed alongside the write-up.

6. Results & Their Significance

Case studies (3,564 files scanned; 550 findings).

Codebase	Files	Findings	F4 type-coupling	Real call sites	Top tier
jjwt 0.13.0	408	193	193	0	CRITICAL
mina-sshd 2.13.1	1,365	323	316	7	CRITICAL
californium 3.14.0	1,001	34	11	23	MEDIUM
shiro 2.2.1	790	0	0	0	—

The headline is *coupling dominance*: **95% of all findings are concrete key-type coupling**, and actual algorithm-selection call sites are rare (30 total, 16 of them only visible through constant propagation). jjwt rates CRITICAL with **zero** visible RSA/EC calls — its migration cost is API-surface churn, not call-site swaps — so the report states plainly that CRITICAL here means "dense key-typed API surface," not "heavy active RSA use." Practically: migration budgets for mature libraries should center on key-typed interfaces, casts, and key-material serialization, with call-site counts read as a lower bound. Methodologically: inventory-style counting (what existing tools produce) measures the *smallest* part of the visible surface. The remaining detection frontier is characterized precisely — enum/registry-based selection (jjwt's `SignatureAlgorithm`) and runtime-dynamic `getInstance(param)` — which is itself a finding about the limits of static crypto discovery.

Benchmarks (read the ratios, not the microseconds — single machine).

Operation	Classical	Hybrid	PQC-only
KEM keygen	43.6 µs	102.4 µs (2.3×)	55.6 µs (1.3×)
KEM encapsulate	108.0 µs	138.2 µs (1.3×)	36.1 µs (0.3×)
KEM decapsulate	62.3 µs	101.2 µs (1.6×)	42.4 µs (0.7×)
Sign	65.6 µs	355.9 µs (5.4×)	513.7 µs (7.8×)
Verify	72.3 µs	171.7 µs (2.4×)	119.9 µs (1.7×)

Three results matter. **(1) The agility abstraction is effectively free:** capability negotiation costs 6.4–9.0 ns and 24 B per operation — about seven orders of magnitude below the crypto it selects — which empirically retires the standard "an agility layer is too slow" objection. **(2) PQC is not uniformly slower; it is differently shaped:** ML-KEM encapsulation/decapsulation are *faster* than X25519 (0.3× / 0.7×) with the cost moved to keygen, while ML-DSA signing is the one genuinely expensive operation (7.8× ECDSA; verification only 1.5–1.7×). Workload shape, not a blanket slowdown, should drive migration planning. **(3) The JVM-specific story is allocation:** hybrid/PQC operations allocate **4–13× more per operation** (hybrid KEM keygen 47.1 KB/op vs. 3.7 KB classical; ML-DSA sign 341.9 KB/op vs. 52.6 KB), i.e., sustained GC pressure under handshake- or signing-heavy load — a managed-runtime effect the C/Rust-centric PQC benchmarking literature does not surface, and the clearest candidate for a standalone empirical contribution.

Scope of claims. All Phase-1 numbers are estimates from a pre-registered heuristic plus microbenchmarks on one machine. The falsifiable claim (H1) remains open by design until Phase 2 supplies measured-effort ground truth; the framework was built so that either outcome — validation or refutation — is publishable.

7. Main Dependencies

Dependency	Version	Role
JavaParser (symbol-solver-core artifact)	3.27.0	AST parsing; used purely syntactically (no classpath)
Bouncy Castle `bcprov-jdk18on`	1.84	All crypto primitives incl. ML-KEM/ML-DSA (multi-release jar; no separate `bcpq` artifact exists)
Jackson Databind	2.18.2	JSON report serialization
JUnit (BOM)	6.1.1	Test framework
JMH (+ shade plugin)	1.37 / 3.6.2	Benchmarks (must run from classpath, not the shaded jar — §3e)

Dependency	Version	Role
JDK / Maven	21 LTS / 3.9+	Toolchain
Python (stdlib only)	3.9+	Phase-2 correlation harness — deliberately zero third-party deps for reproducibility

Two dependency decisions are load-bearing: primitives are *never* hand-implemented (correctness and timing-side-channel risk belong to Bouncy Castle's KAT-tested code; the research contribution is detection, scoring, and composition), and the analysis harness is standard-library-only so the statistics rerun anywhere, byte-for-byte.